BlueShelf Code Walkthrough — the interview answers, in your own code

Companion to NOTES/10 (question bank). Every concept there is shown here in the actual file that implements it, annotated line by line. Numbers like ① ② ③ in the code are explained right below each block. "Answers: Q…" links back to NOTES/10.

---

1. The map — how an AEM project is laid out

```
blueshelf/
├── pom.xml          ← parent: versions, plugins, deploy profiles
├── core/            ← the OSGi BUNDLE: all Java (models, services, servlets, scheduler)
├── ui.apps/         ← content package → /apps/blueshelf  (components, HTL, dialogs,
clientlibs)  [IMMUTABLE]
├── ui.content/      ← content package → /content, /conf   (sample pages, templates, policies)
[MUTABLE]
├── ui.config/       ← content package → OSGi configs per run mode + repoinit
├── ui.frontend/     ← React/TS build → output lands inside ui.apps' clientlib folders
└── all/             ← CONTAINER package embedding all of the above = the ONE deployable
```

Why it matters: Cloud Manager deploys only `all`. `/apps` is immutable at runtime (code ships via pipeline), `/content` & `/conf` are mutable (authors own them). Keeping code and content in separate packages with correct `filter.xml` modes is what prevents a deploy from deleting production content.

Answers: Q35, Q37, Q38.

---

2. One component end-to-end: the Hero

A component = **content node** (properties) + **Java model** (logic) + **HTL script** (markup) + **dialog** (author form). Four files, one feature.

2a. The content (what authors produce)

A node on the page — this is ALL that authoring stores:

```
/content/blueshelf/us/en/jcr:content/root/hero
    sling:resourceType = "blueshelf/components/hero"   ← ①  which component renders me
    jcr:title  = "Back to School TV Deals"
    subtitle   = "Save up to 40% …"
    theme      = "yellow"
    badge      = "Deal of the day"
    cq:styleIds = ["1002","2002"]                       ← ②  Style System selections
```

① `sling:resourceType` is the heart of Sling: the *content* names its renderer. The container just iterates children and includes each one; Sling looks up `/apps/blueshelf/components/hero/hero.html` for this node. ② styles picked by the author in the editor — resolved to CSS classes at render time (chapter 4).

2b. The Sling Model — `core/src/main/java/com/blueshelf/core/models/HeroModel.java`

```
@Model(
    adaptables = {Resource.class, SlingHttpServletRequest.class},   // ①
    resourceType = HeroModel.RESOURCE_TYPE,                         // ②
    defaultInjectionStrategy = DefaultInjectionStrategy.OPTIONAL    // ③
)
@Exporter(name = "jackson", extensions = "json")                    // ④
public class HeroModel {

    public static final String RESOURCE_TYPE = "blueshelf/components/hero";

    @ValueMapValue(name = "jcr:title")   // ⑤
    private String title;

    @ValueMapValue                       // ⑥  field name == property name
    private String subtitle;
    @ValueMapValue private String ctaLabel;
    @ValueMapValue private String ctaLink;
    @ValueMapValue private String theme;
    @ValueMapValue private String badge;

    @SlingObject                         // ⑦
    private Resource resource;

    private String resolvedCtaLink;

    @PostConstruct                       // ⑧
    protected void init() {
        if (StringUtils.isBlank(theme)) theme = "blue";
        if (StringUtils.isNotBlank(ctaLink) && ctaLink.startsWith("/content/")
                && !ctaLink.endsWith(".html")) {
            resolvedCtaLink = ctaLink + ".html";      // ⑨ internal links need .html
        } else {
            resolvedCtaLink = ctaLink;
        }
    }

    public String getTitle()   { return title; }
    public String getCtaLink() { return resolvedCtaLink; }
    public boolean hasCta()    { return StringUtils.isNotBlank(ctaLabel)
                                    && StringUtils.isNotBlank(resolvedCtaLink); }

    @JsonIgnore                          // ⑩
    public String getPath() { return resource != null ? resource.getPath() : null; }
}
```

① **adaptables** — what this model can be created *from*. `Resource` = works anywhere you have a node; `SlingHttpServletRequest` = also works in HTL page rendering and can (in other models) read request state. Declaring both is safe when you don't need the request. Gotcha: a request-ONLY model returns `null` from `resource.adaptTo(...)` — silently. ② binding the model to the resource type enables two things: HTL resolves it automatically and the **Exporter** (④) knows which model renders `hero.model.json`. ③ **OPTIONAL** — without it, ONE missing property (an author left a field empty) throws `MissingElementsException`, the model is null in HTL, the component silently vanishes. We hit this live with `cq:styleIds`. Rule: optional by default, mark truly-required fields explicitly. ④ **Sling Model Exporter**: the same class serves the HTML site AND headless clients — `GET …/hero.model.json` returns this model serialized by Jackson. The JSON is a public API: keep getters stable. ⑤ property name ≠ field name → say it explicitly (`jcr:title`). ⑥ when names match, `@ValueMapValue` alone is enough. Prefer these *specific* injectors over generic `@Inject` — faster, and the source of the value is explicit. ⑦ `@SlingObject` injects framework objects (resource, resolver, response…). ⑧ runs once after injection

— compute derived values HERE, not in getters (HTL may call getters repeatedly). ⑨ the classic link-handling concern: internal content links must end in `.html` for Sling/dispatcher to resolve them; external URLs pass through. (Core Components have a whole LinkHandler for this.) ⑩ excluded from the JSON export — debug info stays out of the public contract.

Answers: Q16, Q17, Q18, Q19, Q20 + the "why not a servlet" follow-up.

2c. The HTL — `ui.apps/.../components/hero/hero.html`

```
<section class="hero hero--${hero.theme} ${style.cssClasses}"       ← ① ⑤
         data-sly-use.hero="com.blueshelf.core.models.HeroModel"    ← ①
         data-sly-use.style="com.blueshelf.core.models.StyleModel"
         data-sly-test="${hero.title}">                              ← ②
    <span class="hero__badge" data-sly-test="${hero.badge}">${hero.badge}</span>
    <h1 class="hero__title">${hero.title}</h1>                       ← ③
    <p class="hero__subtitle" data-sly-test="${hero.subtitle}">${hero.subtitle}</p>
    <a class="hero__cta" data-sly-test="${hero.hasCta}"
       href="${hero.ctaLink @ context='uri'}">${hero.ctaLabel}</a>   ← ④
</section>
<div class="hero hero--empty" data-sly-test="${!hero.title}">Hero: configure a title</div>  ←
⑥
```

① `data-sly-use` instantiates the model by adapting the current request/resource. This is the ONLY place logic enters a template — HTL itself has no string concat, no method calls with args, no `startsWith` (we hit both walls and moved the logic to Java). ② `data-sly-test` = conditional render; false/empty removes the whole element. ③ `${}` output is **auto-escaped for its context** — here HTML-text. This is HTL's security story: XSS-safe by default. ④ `context='uri'` — href escaping; it also blocks `javascript:` URLs. Rich text uses `context='html'` (sanitizes). `context='unsafe'` disables protection = review flag. ⑤ Style System classes from the policy (chapter 4). ⑥ authoring placeholder — rendered only when unconfigured (in AEM proper you'd use `cq:editConfig`'s empty-text; same idea).

Answers: Q21–Q24.

2d. The dialog — `ui.apps/.../hero/_cq_dialog/.content.xml` (trimmed)

```
<jcr:root jcr:primaryType="nt:unstructured" jcr:title="Hero"
          sling:resourceType="cq/gui/components/authoring/dialog">   ← ①
  <content sling:resourceType="granite/ui/components/coral/foundation/container">
    <items>
      <tabs sling:resourceType="granite/ui/components/coral/foundation/tabs">  ← ②
        <items>
          <properties jcr:title="Properties" sling:resourceType=".../container">
            <items>
              <title
sling:resourceType="granite/ui/components/coral/foundation/form/textfield"
                     fieldLabel="Title" name="./jcr:title" required="{Boolean}true"/>   ← ③
              <theme sling:resourceType=".../form/select" fieldLabel="Theme" name="./theme">
                <items>
                  <blue text="Blue" value="blue"/> <yellow text="Yellow" value="yellow"/>
                </items>
              </theme>
            </items>
          </properties>
          <cta jcr:title="Call to action" sling:resourceType=".../container">
            <items>
              <ctaLink sling:resourceType=".../form/pathfield"                          ← ④
                       fieldLabel="CTA link" name="./ctaLink" rootPath="/content/blueshelf"/>
              <newTab sling:resourceType=".../form/checkbox" text="Open in new tab"
                      name="./newTab" value="true" uncheckedValue="false"/>             ← ⑤
            </items>
          </cta>
        </items>
      </tabs>
    </items>
  </content>
</jcr:root>
```

① a dialog is just a **content tree of Granite UI resource types** — AEM renders it as a form (the file name `_cq_dialog` maps to node name `cq:dialog` at package build). ② tabs/containers structure the form; leaf nodes are fields. ③ **`name="./jcr:title"`** is the whole trick: on Done, the editor form does a **Sling POST** to the component node with `./jcr:title=<value>` — dialogs are nothing more than pre-wired Sling POST forms. `@Delete` and `@TypeHint` suffixes handle clearing and typing. ④ `pathfield` = content picker rooted at your site. ⑤ checkbox needs `value`/`uncheckedValue` — otherwise unchecking sends nothing and the old value stays (classic dialog bug).

Answers: Q25, Q26, Q5 + the "authored config vs request state" distinction.

---

3. Inheritance: base components, proxies, overrides

3a. The base ("library") component — `ui.apps/.../base/title/v1/title/`

Full component (script + dialog + model binding), versioned path `v1`, hidden group `.blueshelf.base` so authors never use it directly.

3b. The proxy — `ui.apps/.../components/title/.content.xml` — the ENTIRE file:

```
<jcr:root jcr:primaryType="cq:Component"
    jcr:title="Title"
    sling:resourceSuperType="blueshelf/components/base/title/v1/title"   ← ①
    componentGroup="BlueShelf - Content"/>                                ← ②
```

① **all** behavior — script, dialog, model — is inherited through `sling:resourceSuperType`. Upgrading the site to a v2 title = edit this one line. This is exactly how Core Components are meant to be used (proxy, never copy). ② the proxy owns project-facing identity: title, group (and could add its own clientlib category).

3c. Selective override — `ui.apps/.../components/teaser/teaser.html`:

```
<div class="bs-teaser">                                                    ← ①
    <sly data-sly-include="/apps/blueshelf/components/base/teaser/v1/teaser/teaser.html">
</sly>  ← ②
</div>
```

① same-named file in the proxy **shadows** the inherited script (script resolution walks the supertype chain, nearest wins). ② …but we don't copy-paste the base markup: `data-sly-include` pulls the base *script* in, so the override stays one line of wrapping. `include` = script against current resource; `resource` = full Sling resolution of another node (containers use that).

Answers: Q27, Q23.

---

4. Templates, policies, Style System

4a. The editable template — `ui.content/.../templates/content-page/.content.xml` (trimmed)

```
<jcr:root jcr:primaryType="cq:Template">
  <jcr:content jcr:title="Content Page" status="enabled"
               allowedPaths="[/content/blueshelf(/.*)?]"/>
  <structure jcr:primaryType="cq:Page">                      ← ①
    <jcr:content sling:resourceType="blueshelf/components/page">
      <root sling:resourceType="blueshelf/components/container" editable="{Boolean}true"/>
    </jcr:content>
  </structure>
  <initial jcr:primaryType="cq:Page">                        ← ②
    <jcr:content sling:resourceType="blueshelf/components/page"
                 cq:template="/conf/blueshelf/settings/wcm/templates/content-page">
      <root sling:resourceType="blueshelf/components/container">
        <hero sling:resourceType="blueshelf/components/hero"/>
      </root>
    </jcr:content>
  </initial>
  <policies jcr:primaryType="cq:Page">                       ← ③
    <jcr:content>
      <root cq:policy="blueshelf/components/container/default">   ← ④
        <blueshelf><components>
          <hero cq:policy="blueshelf/components/hero/default"/>   ← ⑤
        </components></blueshelf>
      </root>
    </jcr:content>
  </policies>
</jcr:root>
```

① `structure` = what EVERY page from this template always has (locked layout). ② `initial` = starting content **copied into** each new page. Creating a page = new `cq:Page` node + copy `initial/jcr:content` + set `cq:template` — that IS `PageManager.create()`; our Sites console does those three Sling calls explicitly. ③ `policies` maps components (by position/resource type) to **policy nodes** in `/conf/.../policies/...`. ④ the container's policy decides **which components authors may add** — demo: setting its `components` to one entry shrank the editor's component browser instantly, zero deployment. Governance is content. ⑤ per-component policy → styles and defaults, next.

4b. A policy with Style System — `ui.content/.../policies/blueshelf/components/hero/.content.xml`

```
<default jcr:title="Hero default" sling:resourceType="wcm/core/components/policy/policy">
  <cq:styleGroups>
    <item0 cq:styleGroupLabel="Layout">
      <cq:styles>
        <item0 cq:styleId="1001" cq:styleLabel="Compact"  cq:styleClasses="hero--compact"/>
        <item1 cq:styleId="1002" cq:styleLabel="Centered" cq:styleClasses="hero--center"/>
      </cq:styles>
    </item0>
    <item1 cq:styleGroupLabel="Emphasis">
      <cq:styles>
        <item0 cq:styleId="2001" cq:styleLabel="Dark"     cq:styleClasses="hero--dark"/>
        <item1 cq:styleId="2002" cq:styleLabel="Outlined" cq:styleClasses="hero--outlined"/>
      </cq:styles>
    </item1>
  </cq:styleGroups>
</default>
```

The flow: author ticks styles in the dialog's Styles tab → editor saves `cq:styleIds=["1002","2002"]` on the *component node* → at render, `StyleModel` looks up the policy, translates ids → `"hero--center hero--outlined"` → HTL appends `${style.cssClasses}`. New visual variant = policy entry + CSS. **No Java, no dialog field, no deploy of code.**

4c. The resolver — `core/.../util/Policies.java` (essence)

```
Resource pageContent = pageContentOf(component);           // walk up to the cq:Page's
jcr:content
String template = pageContent.getValueMap().get("cq:template", String.class);   // ①
Resource mapping = rr.getResource(template + "/policies/jcr:content" + rel + "/" +
resourceType); // ②
String policy = mapping.getValueMap().get("cq:policy", String.class);
return rr.getResource("/conf/blueshelf/settings/wcm/policies/" + policy);       // ③
```

① every page knows its template (`cq:template` on jcr:content). ② the template's `policies` tree maps this component (by container-relative path + resource type)… ③ …to the policy node under `/conf`. AEM's `ContentPolicyManager` does exactly this walk; we implemented it to understand it. Real-world gotcha we hit: **`/conf` must be readable by anonymous on publish** (repoinit ACL) — or styles work for admins and vanish for visitors.

Answers: Q28, Q29, Q30, war story 2.

---

5. Backend integration — the heart of "full-stack AEM"

5a. The service interface — `core/.../services/CatalogService.java`

```
@ProviderType
public interface CatalogService {
    ProductPage search(ProductQuery query);        // never throws — degraded page instead
    Optional<Product> getProduct(String sku);
    List<Store> storesNear(String zip);            // empty list when backend down / zip blank
    String status();
}
```

Components depend on THIS, never on HTTP. In tests we register a Mockito mock as the OSGi service — models never touch the network.

5b. The implementation — `core/.../services/impl/CatalogServiceImpl.java` (annotated essence)

```
@Component(service = CatalogService.class, immediate = true)          // ①
@Designate(ocd = CatalogServiceImpl.Config.class)                     // ②
public class CatalogServiceImpl implements CatalogService {

    @ObjectClassDefinition(name = "BlueShelf Catalog Service")
    public @interface Config {                                        // ②
        String baseUrl() default "http://localhost:8081";
        int timeoutMs() default 1500;                                 // ③
        int cacheTtlSeconds() default 60;
        int staleIfErrorSeconds() default 600;
        int breakerFailures() default 3;
        int breakerOpenSeconds() default 20;
    }

    private volatile Config config;

    @Activate @Modified                                               // ④
    protected void activate(Config config) {
        this.config = config;
        this.http = HttpClient.newBuilder()
                .connectTimeout(Duration.ofMillis(config.timeoutMs())).build();
        this.cache.clear();
    }

    // cache-aside with STALE-IF-ERROR, used by every public method:
    private <T> T cached(String key, Class<T> type, Loader<T> loader) {
        Entry e = cache.get(key);
        if (fresh(e))   return markCache((T) e.value);                // ⑤ hit
        if (breakerOpen()) return staleOrNull(e, ...);                // ⑥ fail fast
        try {
            T val = loader.load();                                    //    real HTTP call
            consecutiveFailures.set(0);
            cache.put(key, new Entry(val, now));
            return val;
        } catch (Exception ex) {
            if (consecutiveFailures.incrementAndGet() >= config.breakerFailures())
                openUntil.set(now + config.breakerOpenSeconds() * 1000L);   // ⑥ open breaker
            return staleOrNull(e, ...);                               // ⑦ serve stale
        }
    }
}
```

① registers into the OSGi service registry — `@Reference` / `@OSGiService` anywhere can now inject it. ② **typed config**: the annotation interface generates metatype metadata; values come per environment from `ui.config` (see chapter 11), editable at `/system/console/configMgr` locally. ③ short timeout is the single most important number in the class: the call runs **inside page rendering on a Jetty request thread**. A 30 s hang under load exhausts the pool and takes down EVERY page, not just this component. ④ `@Modified` = config changes re-activate the service **without restart** — deploy a config, behavior changes live. ⑤ TTL cache keyed by an immutable query object (`ProductQuery.cacheKey()`), because publish renders the same lists for thousands of visitors. ⑥ circuit breaker: after N consecutive failures, stop calling for T seconds — a dead backend must not consume threads/timeouts per request. ⑦ **stale-if-error**: an *expired* cache entry is still better than an error page; pages show "prices may be out of date". Three sources reach the UI: LIVE / CACHE / STALE / UNAVAILABLE — and the HTL renders each state.

Verified by chaos drills: kill the API (`failEvery=1` or `docker stop`) → cached pages keep serving, uncached show a friendly message, breaker opens after 3, heals after 20 s.

Answers: Q9–Q12, Q58, Q60 + the resilience follow-up. War story: "chaos test that lied" — fault injection initially didn't cover `/api/stores`; the drill passed without testing anything.

---

6. The component YOU wrote — Store Locator (BLUE-102)

6a. `core/.../models/StoreLocatorModel.java`

```
@Model(adaptables = SlingHttpServletRequest.class,                    // ① request-ONLY
       resourceType = "blueshelf/components/store-locator",
       defaultInjectionStrategy = DefaultInjectionStrategy.OPTIONAL)  // ②
@Exporter(name = "jackson", extensions = "json")
public class StoreLocatorModel {

    @Self private SlingHttpServletRequest request;
    @OSGiService private CatalogService catalogService;
    @SlingObject private Resource resource;

    @ValueMapValue(name = "jcr:title") private String title;
    @ValueMapValue private String defaultZip;

    private String zip;
    private String pageUrl;
    private List<Store> stores = List.of();

    @PostConstruct
    protected void init() {
        String param = request == null ? null :
StringUtils.trimToNull(request.getParameter("zip"));
        zip = param != null ? param : StringUtils.trimToNull(defaultZip);      // ③
        Resource pageContent = Policies.pageContentOf(resource);
        pageUrl = pageContent != null && pageContent.getParent() != null
                ? pageContent.getParent().getPath() + ".html" : null;          // ④
        if (zip != null && catalogService != null) {                           // ⑤
            stores = catalogService.storesNear(zip);
        }
    }
    public boolean isEmpty() { return stores.isEmpty(); }
}
```

① request-only because it reads `?zip=` — and that means `resource.adaptTo(StoreLocatorModel.class)` would be null; only request adaptation works. ② an author may leave Title/Default ZIP empty — must not explode. ③ **precedence: request param beats authored default** — the "content vs request state" distinction: `defaultZip` is CONTENT (dialog, persisted), `?zip=` is REQUEST STATE (per visitor). ④ the **fragment-action trap** fix: inside a component, `resource.path` is the *component node*; a form posting there returns a naked HTML fragment (no page). The model walks up to the `cq:Page` and exposes the page URL — what AEM's `currentPage` gives you. ⑤ null-check the `@OSGiService` too: with OPTIONAL strategy an unregistered service injects null instead of failing — a page must degrade, not NPE.

6b. `ui.apps/.../store-locator/store-locator.html`

```
<section class="stores" data-sly-use.model="com.blueshelf.core.models.StoreLocatorModel">
    <h2 data-sly-test="${model.title}">${model.title}</h2>
    <form class="search" method="get" action="${model.pageUrl @ context='uri'}">   ← ①
        <input type="text" name="zip" value="${model.zip}" required>               ← ②
        <button type="submit">Find stores</button>
    </form>
    <p data-sly-test="${model.empty && model.zip}">No stores found near ${model.zip}. Try
another ZIP.</p>   ← ③
    <p data-sly-test="${model.empty && !model.zip}">Enter a ZIP code to find stores near you.
</p>
    <ul data-sly-test="${!model.empty}">
        <li class="stores__item" data-sly-list.store="${model.stores}">
            <strong>${store.name}</strong><br>
            ${store.address}, ${store.city}, ${store.state} ${store.zip}<br>
            <small>Open ${store.hours}</small>
        </li>
    </ul>
</section>
```

① GET form to the PAGE (see ④ above) — cacheable, bookmarkable, works without JavaScript. ② echoes the effective zip back (param or default). ③ two empty-state lines because **HTL has no string concatenation** — `'near ' + zip` is a compile error by design; split the states (or use `@ format=[…]`).

6c. The service method you implemented

```
@Override
public List<Store> storesNear(String zip) {
    if (zip == null || zip.isBlank()) return List.of();
    String url = config.baseUrl() + "/api/stores?zip=" + enc(zip.trim());
    List<Store> stores = cached("stores|" + zip.trim(), List.class, () -> {
        String body = get(url);
        return body == null ? null
             : mapper.readValue(body, new TypeReference<List<Store>>() {});   // ①
    });
    return stores == null ? List.of() : stores;                               // ②
}
```

① Jackson can't infer generic element types at runtime (erasure) — `TypeReference` is THE idiom for collections. Plus `Store` uses `@JsonIgnoreProperties(ignoreUnknown=true)` so upstream can add fields (`lat`, `lng`) without breaking us. ② the interface's contract: **never throw; empty list = "backend down or nothing to show"** — callers render an empty state, never a 500.

Answers: Q16 (adaptables), Q17, Q6, war story 3, plus the "content vs request state" follow-up.

---

7. Servlets — `core/.../servlets/ProductSearchServlet.java`

```
@Component(service = Servlet.class)
@SlingServletResourceTypes(                          // ①
    resourceTypes = "cq/Page",                       // ②
    selectors = "search",
    extensions = "json",
    methods = "GET")
public class ProductSearchServlet extends SlingSafeMethodsServlet {

    @Reference private CatalogService catalogService;    // ③

    @Override
    protected void doGet(SlingHttpServletRequest request, SlingHttpServletResponse response) {
        ProductQuery q = new ProductQuery(request.getParameter("category"),
                                          request.getParameter("q"), ...);
        ProductPage page = catalogService.search(q);
        response.setHeader("Cache-Control", "private, max-age=30");   // ④
        if (!page.isAvailable()) response.setStatus(503);
        mapper.writeValue(response.getWriter(), page);
    }
}
```

① **resourceType-bound** (recommended) vs path-bound `/bin/...`: the request resolves through the content first, so **ACLs apply**, the URL only exists where a page exists, and the dispatcher can reason about it per-path. URL: `/content/blueshelf/us/en.search.json?q=oled`. ② gotcha we hit: a page NODE's type is `cq:Page` (the page *component* type lives on `jcr:content`), so page-level selector servlets bind to `cq/Page`. ③ `@Reference` = service injection in DS components (models use `@OSGiService`). ④ search JSON is query-dependent → tell every cache layer not to share it. Path-bound counter-example in the repo: `ReplicationServlet` on `/bin/blueshelf/replicate` — author-only admin action, must check inputs itself, dispatcher denies `/bin/*` on publish.

Answers: Q4, Q1, Q2.

---

8. Replication + dispatcher flush — `core/.../services/impl/ReplicationServiceImpl.java` (essence)

```
// activate = serialize the page subtree and push it to publish over HTTP
post(parentPath, Map.of(
    ":operation", "import", ":contentType", "json",
    ":name", pageName, ":content", jsonOfSubtree,
    ":replace", "true", ":replaceProperties", "true"));          // ①

// then tell the caches:
private void flush(String path, String action) {
    for (String url : config.dispatcherFlushUrls()) {
        HttpRequest req = HttpRequest.newBuilder(URI.create(url))
            .header("CQ-Action", action)          // ② Activate | Deactivate
            .header("CQ-Handle", path)            // ②  the content path
            .POST(HttpRequest.BodyPublishers.noBody()).build();
        http.send(req, ...);                      //   errors LOGGED, not thrown  ③
    }
    // optional webhook → Next.js /api/revalidate (ISR tag invalidation)   ④
}
```

① author and publish are **separate repositories**; activation transfers content (AEM: replication agents / Sling Content Distribution — same author→HTTP→publish shape). ② the real dispatcher flush protocol: POST with `CQ-Action` / `CQ-Handle` headers; the dispatcher deletes the page's cache files and touches `.stat` files. **Order matters: content first, THEN flush** — flush-first lets the next visitor re-cache the OLD page. ③ a dead cache layer must never break authoring — log and continue. ④ the same event drives the React storefront's cache invalidation — one publish, every cache layer notified. War story 4 lives here: content replicated but component *code* not deployed to publish → published page rendered a raw node dump (Sling's HtmlRenderer fallback). Content replicates; code deploys — to every tier.

Answers: Q39, Q42, war story 4.

---

9. The dispatcher — `dispatcher/dispatcher.any` (annotated)

```
/filter {
  /0001 { /type "deny"  /url "*" }                                        ← ① deny by default
  /0010 { /type "allow" /method "GET" /path "/content/blueshelf/*" /extension '(html|json)' }
  /0030 { /type "deny"  /url "/system/*" }   /0031 { /type "deny" /url "/bin/*" }
  /0034 { /type "deny"  /url "*.infinity.json" }                          ← ② no repo dumps
  /0040 { /type "deny"  /method "POST" /url "*" }                         ← ③ repo is writable —
never from the internet
}
/cache {
  /docroot "/var/cache/dispatcher"
  /statfileslevel "4"                                                      ← ④
  /serveStaleOnError "1"                                                   ← ⑤
  /rules {
    /0000 { /glob "*" /type "deny" }                                       ← ⑥ cache allow-list
    /0001 { /glob "/content/blueshelf/*.html" /type "allow" }
    /0011 { /glob "/content/blueshelf/*/search.html" /type "deny" }        ← ⑦ query-dependent:
NEVER
  }
  /ignoreUrlParams {
    /0001 { /glob "*" /type "deny" }                                       ← ⑧ unknown params →
uncacheable
    /0002 { /glob "utm_*" /type "allow" }                                  ←   known-irrelevant →
ignored
  }
  /allowedClients { /0000 { /glob "*" /type "deny" } /0002 { /glob "172.*" /type "allow" } }
← ⑨
}
```

① security model: allow-list. Everything not explicitly allowed is 404. ② default JSON renderers (`.infinity.json`, `.2.json`) dump the repository — deny. ③ the Sling POST servlet writes the repo; publish must never accept POST from outside. ④ on flush, `.stat` files are touched down to depth 4; cached files older than an ancestor `.stat` are refetched. Depth 0 = one activation invalidates the whole site (thundering herd); too deep = stale shared navs. Pick ≈ language-root depth. ⑤ publish down → serve stale copies instead of 502s (verified by stopping publish). ⑥⑦⑧ **ALL rule sets are last-match-wins.** Our production-style incident: a trailing `allow *.html` silently overrode ⑦, and an `ignoreUrlParams allow *` made `?q=` irrelevant → the cache file is keyed by PATH only → first user's search results served to everyone. Fix + a CI validator that simulates last-match-wins and asserts the final VERDICT (a deny line existing means nothing if something later overrides it). ⑨ who may POST the flush — otherwise anyone can empty your cache (a cheap DoS).

Answers: Q40–Q44, war story 5.

---

10. Headless — from exporter to React

10a. The page JSON contract — `core/.../models/PageModel.java` (essence)

```
@Model(adaptables = SlingHttpServletRequest.class,
       resourceType = {"blueshelf/components/page", "cq/Page"},    // ① page URL itself
answers
       defaultInjectionStrategy = OPTIONAL)
@Exporter(name = "jackson", extensions = "json")
public class PageModel {
    @OSGiService private ModelFactory modelFactory;                // ②

    @PostConstruct protected void init() {
        for (Resource child : content.getChildren()) {
            items.put(child.getName(), ComponentExporter.export(request, child,
modelFactory)); // ③
            order.add(child.getName());
        }
    }
    @JsonProperty(":items")      public Map<String,Object> getItems()      { ... }
    @JsonProperty(":itemsOrder") public List<String>       getItemsOrder() { ... }
    @JsonProperty(":type")       public String             getType()       { ... }   // ④
}
```

① `GET /content/blueshelf/us/en.model.json` — the headless entry point. ② `ModelFactory` resolves each child's registered model dynamically (wrapping the request so request-scoped models — search's `?q=`, PDP's suffix — work in JSON too). ③ every child serializes with its own model; components without a model export raw properties (never break the feed). ④ `:type` / `:items` / `:itemsOrder` = the **AEM SPA-editor contract** — the same shape `aem-spa-page-model-manager` consumes.

10b. The React side — `storefront/src/components/aem/mapping.tsx`

```
const registry: Record<string, ComponentType<any>> = {   // ① MapTo(...) as a plain map
  'blueshelf/components/hero': Hero,
  'blueshelf/components/product-list': ProductList,
  ...
};
export function AemComponent({ item }: { item: AemItem }) {
  const Cmp = registry[item[':type']];
  if (!Cmp) return <div>Unmapped component: <code>{item[':type']}</code></div>;  // ②
  return <Cmp {...item} />;
}
```

① `:type` → React component — identical in spirit to AEM SPA SDK's `MapTo('blueshelf/components/hero')(Hero)`. ② unknown type renders a placeholder — authors adding a component the frontend doesn't know must not break the page.

10c. Real AEMaaCS GraphQL — `storefront/src/app/frescopa/page.tsx`

```
// Preferred: published PERSISTED QUERY (cacheable GET):
//   GET <publish>/graphql/execute.json/<config>/<name>
// Fallback (non-prod only): direct POST endpoint
const POST_ENDPOINT = 'https://publish-p153710-e1614654.adobeaemcloud.com'
                    + '/content/_cq_graphql/aem-boilerplate-frescopa/endpoint.json';
const LIST_QUERY = '{ articleList(limit: 20, sort: "title ASC") { items { _path title author }
} }';
```

The lesson encoded here: prod exposes **only persisted queries** (cacheable, allow-listed); arbitrary POST GraphQL stays open only on sandboxes. Filter syntax to recall: `filter: {author: {_expressions: [{value: "Emma Davis"}]}}`; variations via `variation: "teaser"`; images as `... on ImageRef { _publishUrl }`.

Answers: Q19, Q45–Q49.

---

11. Config per environment — `ui.config`

```
ui.config/.../osgiconfig/
├── config/            org.apache.sling.jcr.repoinit.RepositoryInitializer~blueshelf.cfg.json
← ①
├── config.author/     com.blueshelf...ReplicationServiceImpl.cfg.json      ← agents only on
author
├── config.publish/    ...ReplicationServiceImpl.cfg.json ("enabled": false)
├── config.local/      ...CatalogServiceImpl.cfg.json  (baseUrl http://catalog-api:8081)
├── config.prod/       ...CatalogServiceImpl.cfg.json  (tighter timeout, longer stale window)
└── config.author.prod/ ...ReplicationServiceImpl.cfg.json                  ← ② combined run
modes
```

```
// config.author.prod/...ReplicationServiceImpl.cfg.json
{
  "publishUrl": "http://publish:8080",
  "password": "$[env:AEM_ADMIN_PASSWORD;default=admin]",     ← ③
  "dispatcherFlushUrls": ["http://dispatcher:8080/dispatcher/invalidate.cache"]
}
```

① **repoinit** — idempotent startup scripts (create paths, service users, ACLs). The sanctioned way; it fixed our "anonymous can't read /conf" incident. ② run-mode folders combine (`author` AND `prod`); most specific wins. ③ Felix config interpolation — secrets from environment variables, same `$[env:…]` syntax AEMaaCS uses. The **config-precedence demo** to retell: console-edit a value → it shadows this file; redeploying does NOT reclaim it (the installer won't overwrite what it didn't create); deleting the console config restores it. Why AEMaaCS removed the console.

Answers: Q12, Q55, war story 6.

---

12. Scheduler — `core/.../services/impl/CatalogPrewarmScheduler.java`

```
@Component(service = Runnable.class, property = {       // ① whiteboard: register, don't call
        "scheduler.period:Long=300",
        "scheduler.immediate:Boolean=true",
        "scheduler.concurrent:Boolean=false"            // ② never overlap
})
public class CatalogPrewarmScheduler implements Runnable {
    @Reference private CatalogService catalogService;
    @Override public void run() {
        for (String category : new String[]{"tvs", "laptops"})
            catalogService.search(ProductQuery.category(category, 6));   // ③
    }
}
```

① the Sling Commons Scheduler (Quartz) discovers any `Runnable` service carrying `scheduler.*` properties — the OSGi *whiteboard pattern*. Cron alternative: `scheduler.expression = "0 0/5 * * * ?"`. ② overlapping runs on a slow backend = thread pile-up. ③ keeps hot queries warm so the first visitor after TTL expiry never pays backend latency. AEMaaCS gotchas to say: schedulers run on EVERY instance (gate by run mode / `scheduler.runOn`); use **Sling Jobs** when execution must be guaranteed exactly-once-ish.

Answers: Q14.

---

13. Testing — how AEM code is actually tested

13a. Model test — `core/src/test/.../HeroModelTest.java`

```
@ExtendWith(SlingContextExtension.class)                        // ①
class HeroModelTest {
    private final SlingContext context = new
SlingContext(ResourceResolverType.RESOURCERESOLVER_MOCK);

    @BeforeEach void setUp() {
        context.addModelsForClasses(HeroModel.class);           // ② no classpath scan in mocks
        context.load().json("/com/.../HeroModelTest.json",      // ③ JSON fixture = repo
content
                            "/content/blueshelf/us/en");
    }

    @Test void adaptsFromResourceAndResolvesInternalLink() {
        context.currentResource(".../root/hero");
        HeroModel hero = context.request().adaptTo(HeroModel.class);   // ④
        assertEquals("/content/blueshelf/us/en/tvs.html", hero.getCtaLink());  // .html
appended
    }
}
```

① wcm.io mocks: in-memory repo + resolver + Sling Models registry (AEM projects use `AemContext`, a superset adding Page/PageManager/DAM). ② register your model classes explicitly. ③ fixtures are JSON shaped exactly like repository content. ④ test through the same adaptation path production uses.

13b. Service test with OSGi config — `CatalogServiceImplTest.java`

```
service = context.registerInjectActivateService(new CatalogServiceImpl(), Map.of(   // ①
    "baseUrl", stubServerUrl, "timeoutMs", 1000, "cacheTtlSeconds", 60,
    "breakerFailures", 2, "breakerOpenSeconds", 60));
...
service.search(q);  service.search(q);
assertEquals(1, calls.get(), "backend called once — second hit from cache");        // ②
```

① activates the component with a typed config map — exactly what OSGi does at runtime, so `@Activate` runs for real. HTTP is stubbed with a throwaway `HttpServer`. ② prove the cache with a call counter, prove the breaker by asserting the backend is NOT called after it opens, prove stale-if-error by asserting `Source.STALE`. Services get behavioral tests, not just parsing tests.

13c. Servlet test — `ServletsTest.java`

```
context.registerService(CatalogService.class, mock(CatalogService.class));  // deps as mocks
ProductSearchServlet servlet = context.registerInjectActivateService(new
ProductSearchServlet());
context.request().setParameterMap(Map.of("q", "tv"));
servlet.doGet(context.request(), context.response());
assertEquals(200, context.response().getStatus());
```

27 tests, 90 % line coverage — above Cloud Manager's 50 % gate.

Answers: Q56, Q57.

---

14. CI — `.github/workflows/ci.yml` as a Cloud Manager mirror

```
- name: Build, unit tests, package validation, AEM analyser
  working-directory: blueshelf
  run: mvn -B -ntp clean install -Daem.analyser.failOnErrors=true     # ①
- name: Coverage gate (JaCoCo >= 50% lines on core)                   # ②
```

① one Maven run gives three Cloud Manager gates: unit tests, **FileVault package validation** (filter/type/overlap errors — we hit six of them and can name each), and **`aemanalyser-maven-plugin`** — Adobe's real Cloud Manager analysers running against the latest AEMaaCS SDK. Proven by breaking it deliberately: importing an implementation package produced the exact pipeline error — `[api-regions-exportsimports] Bundle blueshelf.core is importing package(s) org.apache.sling.models.impl … but no bundle is exporting these`. ② Cloud Manager's coverage metric, enforced the same way. Then: dispatcher config validation (our semantic validator), Docker images to GHCR, deploy workflow, and a static publish of the storefront to GitHub Pages.

Answers: Q53, exercise B4's war story.

How to study this document

Pick a chapter → read the code block → cover the annotations → explain each ① ② ③ aloud → check yourself. If you can do that for chapters 2, 5, 6 and 9, you can survive any whiteboard session this role will throw at you — because you're not reciting AEM trivia, you're describing **your own code**.